1. **Creating a Custom Dataset for your files**

   A custom Dataset class must implement three functions: **init**, **len**, and **getitem**. Take a look at this implementation; the SatelliteImageClassification images are stored in a directory img_dir, and their labels are stored separately in a CSV file annotations_file.

   __init__

   The *_init_* function is run once when instantiating the Dataset object. We initialize the annotations file, and both transforms

   __len__

   The *_len_* function returns the number of samples in our dataset.

   __getitem__

   The *_getitem_* function loads and returns a sample from the dataset at the given index idx. Based on the index, it identifies the image's location on disk, converts that to a tensor using read_image, retrieves the corresponding label from the csv data in self.img_labels, calls the transform functions on them (if applicable), and returns the tensor image and corresponding label in a tuple.

```
import os
import pandas as pd
from torch.utils.data import Dataset
from PIL import Image

classes = ('apple', 'avocado', 'banana', 'cherry', 'kiwi', 'mango', 'orange', 'pinenapple', 'strawberries', 'watermelon')
idx_to_class = {i:j for i, j in enumerate(classes)}
class_to_idx = {value:key for key,value in idx_to_class.items()}


class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 1], self.img_labels.iloc[idx, 0])
        label = self.img_labels.iloc[idx, 1]

        # Load image
        image = Image.open(img_path).convert('RGB')
        # transform to number
        label = class_to_idx[label]

        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```

- Preparing your data for training with DataLoaders

```
import torchvision.transforms.v2 as transforms
import torch
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder

transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToImage(),  # Convert to image format
    transforms.ConvertImageDtype(torch.float32),  # Convert image dtype
    transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))])


train_dataset = CustomImageDataset(annotations_file="./FruitClassification/train/annotation.csv", img_dir="./FruitClassification/train", transform=transform)
train_dataloader = DataLoader(train_dataset, batch_size=4, shuffle=True)
```

- Show sample image from the DataLoader

```
import matplotlib.pyplot as plt
import numpy as np
import torchvision


def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()


# get some random training images
dataiter = iter(train_dataloader)
images, labels = next(dataiter)

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(4)))
```

Show hidden output

2. Define a Convolutional Neural Network

```
from torchvision.models import resnet50, ResNet50_Weights
import torch.nn as nn

# New weights with accuracy 80.858%
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
model.fc = nn.Linear(2048, len(classes))
print(model)
```

Show hidden output

3. Define a Loss function and optimizer

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

4. Train the network

```
torch.manual_seed(0)

# Check if GPU (cuda) is available
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# Move our model to device (GPU or CPU)
model.to(device)
model.train()

num_epochs = 4

for epoch in range(num_epochs):  # loop over the dataset multiple times
    running_loss = 0.0

    for batch_idx, (inputs, labels) in enumerate(train_dataloader):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = inputs.to(device), labels.to(device)

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if batch_idx % 500 == 499:    # print every 200 mini-batches
            print(f'Epoch [{epoch + 1}/{num_epochs}], Batch [{batch_idx + 1}/{len(train_dataloader)}], Loss: {running_loss / 500:.4f}')
            running_loss = 0.0

print('Finished Training')
```

Show hidden output

- Save the model that we have trained

```
# Save model
torch.save(model.state_dict(), 'model.pt')
```